

Universidad Técnica Particular de Loja



Informe Técnico: Sistema de Gestión de Activos "Lab-Inventario"

Autores: López Díaz Jordan Gerardo, Escudero Mena Mateo Sebastián

Carrera: Computación

Asignatura: Programación Orientada a Objetos

Docente: Ing. Bustamante Granda Wayner Xavier

Paralelo: D

Universidad: Universidad Técnica Particular de Loja

Fecha: 24-07-2026

Loja-Ecuador

1. Introduccion

Lab-Inventario es una aplicación de consola desarrollada en Java, diseñada para la gestión integral de activos dentro de entornos de laboratorio. El sistema surge como respuesta al descontrol manual de equipos e insumos, permitiendo una transición hacia un registro centralizado y automatizado. El objetivo principal es optimizar el control de activos, garantizando la persistencia de datos y reduciendo los errores humanos en la administración de inventario.

2. Arquitectura y diseño del sistema

La solución adopta una arquitectura por capas (Vista, Controlador, Servicio, Repositorio) que separa la interacción del usuario, la lógica de negocio y el acceso a datos. Esta estructura es fundamental para reducir el acoplamiento y asegurar la mantenibilidad del software.

2.1. Principios SOLID aplicados

El desarrollo del sistema se fundamenta en los cinco principios SOLID para garantizar un código escalable y robusto:

- **Responsabilidad Única (SRP):** Cada clase posee una razón única de cambio; por ejemplo, las clases de vista gestionan solo la interacción, mientras que los repositorios se limitan a la persistencia.
- **Abierto/Cerrado (OCP):** El sistema permite la incorporación de nuevos tipos de activos (Hardware, Periférico, Licencia) sin modificar el código fuente

existente, gracias a la implementación polimórfica de métodos como `calcularCostoMantenimiento()`.

- **Sustitución de Liskov (LSP):** Las subclases son intercambiables en cualquier contexto que espere un tipo `Activo`, asegurando la corrección del programa mediante el uso de listas polimórficas.
- **Segregación de Interfaces (ISP):** Se definieron interfaces específicas (ej. `ActivoRepository`, `MantenimientoRepository`) para evitar que los clientes dependan de operaciones innecesarias.
- **Inversión de Dependencias (DIP):** Los controladores y servicios dependen de abstracciones (interfaces), delegando la inyección de las implementaciones concretas a la clase `Main`.

3. Funcionalidad e innovación

El sistema destaca por su capacidad de realizar un CRUD completo (registrar, listar, actualizar, eliminar) desde la consola, con persistencia segura mediante JDBC y SQLite.

3.1. Innovaciones tecnologicas

Como valor agregado, el sistema integra:

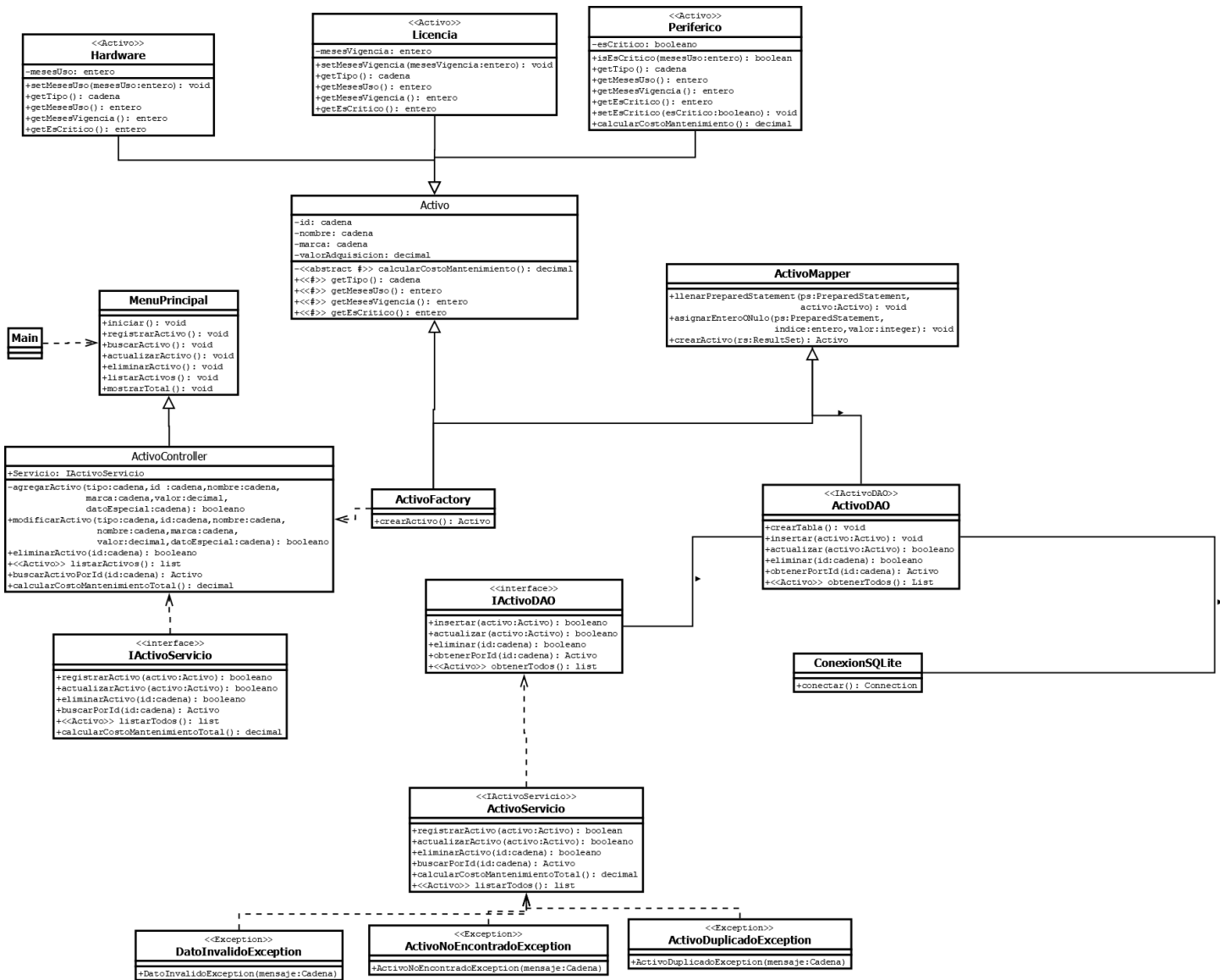
- **Cálculo de costos y depreciación:** Automatización del cálculo de mantenimiento y depreciación de activos en tiempo real.
- **Alertas Preventivas:** Un motor de alertas que procesa reglas de negocio antes de guardar los estados finales en la base de datos.

4. Patrones de Diseño implementados

Para lograr el proyecto mencionada, se aplicaron patrones de diseño estándar:

- Repository: Abstrae la persistencia de datos, permitiendo que la capa de servicio sea agnóstica al motor de base de datos utilizado.
- Factory Method: Encapsula la creación de subtipos de activos a partir de los datos leídos de la base de datos, evitando que el repositorio conozca detalles de instanciación.
- Singleton: Aplicado en la clase ConexionSQLite para asegurar que exista una única conexión activa y gestionada eficientemente.
- Inyección de Dependencias: Utilizada para ensamblar el grafo de objetos desde la clase Main, facilitando el desacoplamiento total.

5. Diagrama UML



6. Enlaces de video

<https://youtu.be/9QR4yYSSKL0>

7. Conclusiones

En conclusión, El proyecto ha resultado en un sistema funcional, persistente y robusto que cumple con los objetivos planteados inicialmente. El mayor reto técnico fue la integración de JDBC con una arquitectura modular y principios SOLID, lo cual fortaleció las competencias de diseño de software del equipo.